

Pixly Project Overview

Executive Summary

Pixly is an **AI-powered image transformation service** that automatically processes images to meet specific compliance requirements. Instead of manually configuring image settings, users simply describe what they need in natural language (like "Image must be JPEG, max 200KB, minimum resolution 600x600"), and the AI generates and executes the necessary image transformations.

Problem Statement

Many applications have strict image requirements regarding file format, size, dimensions, and quality. Meeting these requirements typically requires:

- Technical knowledge of image processing tools
- Manual editing in photo editing software
- Understanding of complex FFmpeg commands

Pixly eliminates these barriers by using AI to interpret requirements and automatically transform images.

Architecture Overview

Monorepo Structure

The project follows a monorepo architecture with two main components:

- **Frontend:** `imagefit-ai/` - User interface for uploading images and defining requirements
- **Backend:** `imagefitai-backend/` - API that orchestrates the AI-powered transformation pipeline

Key Technologies

Frontend Stack

Technology	Purpose
React 19	Modern UI library for building interactive interfaces
TypeScript	Type-safe JavaScript for robust frontend development
Vite	Fast build tool and development server
Axios	HTTP client for API communication
AWS S3 Direct Upload	Efficient file handling using presigned URLs

Backend Stack

Technology	Purpose
Django 4.2	Python web framework for robust API development
Django REST Framework	Toolkit for building Web APIs
PostgreSQL	Production database (SQLite for development)
AWS S3	Scalable cloud storage for images
AWS Bedrock + Claude 3.5 Haiku	AI model for generating FFmpeg commands from natural language
FFmpeg	Industry-standard multimedia processing tool
Gunicorn	Production-grade WSGI HTTP server
WhiteNoise	Efficient static file serving

Infrastructure

Technology	Purpose
Render	Cloud deployment platform
PostgreSQL (Managed)	Database hosting service

How It Works - Processing Pipeline

1. Upload Phase

- User uploads an image through the frontend interface
- Frontend requests a presigned S3 URL from the backend

- Image uploads directly to AWS S3 (bypassing backend for efficiency)

2. Requirements Input

- User describes transformation requirements in plain English
- **Examples:**
 - "Convert to JPEG, compress to under 200KB, ensure minimum 600x600 resolution"
 - "Make it PNG format, resize to 1200x1200, reduce file size"
 - "Convert to WebP, optimize for web, maximum 150KB"

3. AI Processing Pipeline (Backend)

Step-by-step process:

1. **Download:** Image is downloaded from S3 to temporary workspace (`/tmp/imagefitai/job-{id}/`)
2. **Metadata Extraction:** FFprobe extracts current image properties (dimensions, format, size, codec)
3. **AI Generation:** Metadata + user requirements sent to **Claude AI via AWS Bedrock**
4. **Command Generation:** Claude analyzes requirements and generates specific FFmpeg commands
5. **Validation:** Commands are validated for security (prevent injection attacks)
6. **Execution:** FFmpeg commands execute in isolated environment with 30-second timeout
7. **Upload:** Transformed image uploaded back to S3 (`outputs/{job_id}/`)
8. **Cleanup:** Temporary files and directories are removed

4. Results Delivery

- Frontend polls job status endpoint every 1 second (up to 60 seconds)
- User sees before/after image comparison
- Presigned download URLs provided for both original and transformed images
- Summary of transformations applied is displayed

API Endpoints

Base URL: `/api/`

1. Generate Presigned Upload URL

POST /api/uploads/presign

Request Body:

```
{
  "fileName": "example.jpg",
  "contentType": "image/jpeg"
}
```

Response:

```
{
  "uploadUrl": "https://s3.amazonaws.com/...",
  "s3Key": "uploads/unique-key.jpg"
}
```

2. Create Transformation Job

POST /api/jobs

Request Body:

```
{
  "s3Key": "uploads/unique-key.jpg",
  "rulesText": "Convert to JPEG, max 200KB, minimum 600x600"
}
```

Response:

```
{
  "jobId": "123",
  "status": "processing"
}
```

3. Get Job Status

GET /api/jobs/<job_id>

Response:

```
{
  "jobId": "123",
  "status": "completed",
  "summary": "Image converted to JPEG, compressed to 180KB, resized to 800x800",
  "originalImageUrl": "https://s3.amazonaws.com/...",
  "outputImageUrl": "https://s3.amazonaws.com/..."
}
```

Security Features

Command Validation

- FFmpeg commands must start with **ffmpeg**



- Prevents absolute path access (except for input files)
- No arbitrary command execution possible

Isolated Execution

- Each job runs in its own temporary directory
- No access to system files outside workspace
- Complete cleanup after job completion

Timeout Protection

- All FFmpeg commands limited to 30-second execution time
- Prevents resource exhaustion attacks

Production Security

- CORS protection configured
- CSRF token validation
- Secure headers enabled
- Environment-based secrets management

Database Model

Job Model

Tracks the entire lifecycle of an image transformation:

Field	Type	Description
`id`	Integer	Primary key
`s3_key`	String	Input image S3 location

`rules_text`	Text	User's natural language requirements
`status`	String	pending → processing → completed/failed
`original_metadata`	JSON	Image properties extracted by FFprobe
`constraints`	JSON	Parsed requirements from Claude
`commands`	JSON	FFmpeg commands generated by Claude
`output_s3_key`	String	Transformed image S3 location
`summary`	Text	Human-readable description of changes
`error`	Text	Error message if job failed
`created_at`	DateTime	Job creation timestamp
`updated_at`	DateTime	Last update timestamp

Deployment Architecture

Render Cloud Platform

Components:

- **PostgreSQL Database** (`pixly-db`) - Managed database service
- **Web Service** (`pixly-backend`) - Gunicorn-powered Django application
- **AWS S3** - External storage for images

Build Process:

1. Install Python dependencies from `requirements.txt`
2. Run `collectstatic` to gather static files
3. Execute database migrations
4. Start Gunicorn server

Environment Configuration:

- Production: `DEBUG=False`, PostgreSQL via `DATABASE_URL`
- CORS and CSRF configured for frontend domain
- AWS credentials for S3 and Bedrock access
- FFmpeg installed on production server

Development Workflow

Backend Development

```
cd imagefitai-backend
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver
# Access at http://localhost:8000
```

Frontend Development

```
cd imagefit-ai
npm install
npm run dev
# Access at http://localhost:5173
```

Demo Script - Talking Points

Opening (30 seconds)

"Today I'm presenting Pixly, an AI-powered image transformation service that makes image compliance effortless. Whether you're building a profile system, e-commerce platform, or any application with image requirements, Pixly eliminates the technical complexity."

Problem Statement (30 seconds)

"Many applications require images to meet specific criteria - certain formats, file sizes, or dimensions. This usually requires manual editing or technical knowledge of tools like FFmpeg. Users shouldn't need to be image processing experts to meet these requirements."

Solution Overview (45 seconds)

"Pixly lets users describe what they need in plain English - like 'Convert to JPEG, compress to under 200KB, ensure minimum 600x600 resolution' - and AI automatically handles the technical transformation. The backend uses Claude AI through AWS Bedrock to interpret requirements and generate FFmpeg commands, which are then executed securely to transform the image."

Technical Architecture (60 seconds)

"The application is built as a modern monorepo with:

- A React TypeScript frontend for the user interface
- A Django REST backend that orchestrates the processing pipeline

- AWS S3 for scalable image storage
- AWS Bedrock with Claude 3.5 Haiku for AI-powered command generation
- FFmpeg for the actual image transformations

The architecture ensures security through command validation, isolated execution environments, and timeout protection. It's production-ready and deployed on Render with automated builds and migrations."

Live Demo (2-3 minutes)

"Let me show you how it works:

1. [Upload image] I'll start by uploading this image
2. [Enter requirements] Now I'll describe what I need in natural language
3. [Submit] The backend is now downloading the image, extracting metadata, sending it to Claude AI to generate FFmpeg commands, executing those commands, and uploading the result
4. [Show results] Here we can see the before and after images, along with a summary of what was changed
5. [Download] And I can download the transformed image"

Closing (30 seconds)

"Pixly demonstrates how AI can make complex technical operations accessible to everyone. By combining modern web technologies with large language models, we've created a service that's both powerful and user-friendly. The project is fully deployed and ready for production use."

Future Enhancements

Potential Features

- **Batch Processing:** Transform multiple images at once
- **Async Processing:** Background job queue for large images
- **Template Library:** Save and reuse common transformation requirements
- **Advanced Previews:** Real-time preview before final processing
- **API Access:** Allow developers to integrate Pixly into their applications
- **Usage Analytics:** Track popular transformations and optimize prompts
- **Additional Formats:** Support for video transformations

- **Custom Watermarks:** Add branding to processed images

Project Statistics

- **Languages:** Python, TypeScript/JavaScript
- **Total Components:** Frontend App + Backend API + AI Integration
- **Database Tables:** 1 primary (Job model)
- **API Endpoints:** 3 main endpoints
- **Cloud Services:** AWS S3, AWS Bedrock, Render
- **External Dependencies:** 20+ Python packages, 15+ npm packages

Conclusion

Pixly represents a practical application of AI in solving real-world problems. By leveraging large language models for command generation and combining it with robust backend processing, we've created a service that democratizes image processing. The project showcases modern full-stack development practices, cloud architecture, and AI integration.

Project Repository: [Your GitHub URL]

Live Demo: [Your Render URL]

Contact: [Your Contact Information]

Document generated for project demonstration purposes